

# TP2 — Pipeline data temps réel & plateforme data

Prolongement du TP1 — Immobilier × Énergie, Loire-Atlantique (44)

Louis Valette

Alexis Fouquet

Ruben Cofflard

## Contents

|                                         |   |
|-----------------------------------------|---|
| 1. Architecture                         | 1 |
| 2. Sources                              | 2 |
| 3. Data lake                            | 2 |
| 4. Agrégation et nettoyage              | 3 |
| 5. Observabilité (Prometheus + Grafana) | 3 |
| 6. Data visualization                   | 3 |
| 7. Vérification                         | 4 |

## 1. Architecture

Le TP1 a construit une base PostgreSQL statique (import unique, département 44) croisant DVF et DPE. Le TP2 ajoute un pipeline **temps réel** rechargeant la même base en continu : Kafka, data lake brut, Spark Structured Streaming, PostgreSQL, monitoring (Grafana) et dataviz métier (Metabase, outil dédié).

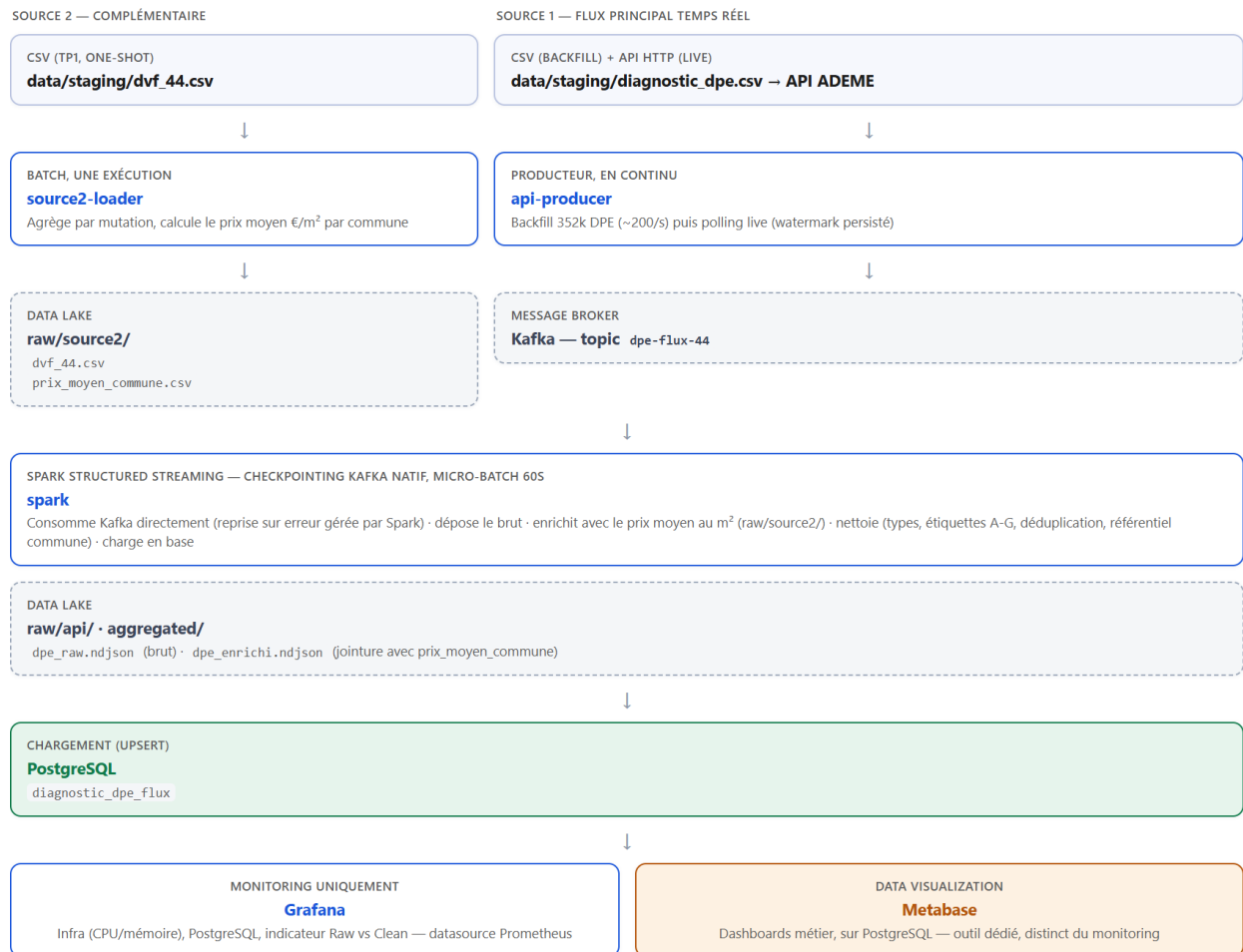


Figure 1: Architecture du pipeline temps réel TP2

## 2. Sources

| # | Source                                        | Nature                                            | Rôle                                                    |
|---|-----------------------------------------------|---------------------------------------------------|---------------------------------------------------------|
| 1 | API ADEME<br>dpe03existant                    | API HTTP, alimentée sur<br>Kafka par api-producer | Flux principal temps réel                               |
| 2 | DVF (département 44, déjà<br>utilisée en TP1) | CSV, chargée en un lot par<br>source2-loader      | Enrichit le flux 1 (prix<br>moyen au m² par<br>commune) |

L'API ADEME ne produit que quelques nouveaux DPE par semaine pour le seul département 44 — insuffisant pour une démo courte. api-producer rejoue donc d'abord les 352k DPE du TP1 à débit maîtrisé (200/s, source\_flux=backfill), puis bascule en interrogation continue de l'API réelle (source\_flux=live, watermark persisté). Le backfill ne s'exécute qu'une fois (/state/backfill\_done).

## 3. Data lake

MinIO (S3, proposé par le sujet) a fermé ses images Docker officielles derrière une authentification (Docker Hub et quay.io renvoient 401), incompatible avec un docker compose up reproductible sans compte préalable. Le

data lake est un volume Docker nommé, organisé selon l'arborescence de l'énoncé :

|              |                                    |                                 |
|--------------|------------------------------------|---------------------------------|
| raw/api/     | dpe_raw.ndjson                     | – flux Kafka brut, non filtré   |
| raw/source2/ | dvf_44.csv, prix_moyen_commune.csv |                                 |
| aggregated/  | dpe_enrichi.ndjson                 | – flux enrichi, avant nettoyage |

## 4. Agrégation et nettoyage

spark/jobs/clean\_and\_load.py consomme le topic Kafka dpe-flux-44 directement en Spark Structured Streaming (micro-batch 60s, checkpointing Kafka natif). Par micro-batch :

1. dépôt du brut dans raw/api/ ;
2. **enrichissement** : jointure avec le prix moyen au m<sup>2</sup> de la commune (calculé par source2-loader, même correction qu'en TP1 : agrégation par mutation avant calcul du prix/m<sup>2</sup>) → aggregated/ ;
3. **nettoyage** : types, étiquettes DPE/GES dans A-G, déduplication sur numero\_dpe, rejet des communes hors référentiel (FK vers commune) ;
4. upsert dans diagnostic\_dpe\_flux ([sql/06\\_create\\_flux\\_table.sql](#), ON CONFLICT DO NOTHING).

Le connecteur Kafka de Spark est résolu et figé dans l'image au build, pas téléchargé au démarrage : docker compose up -d ne dépend d'aucun accès réseau à l'exécution.

## 5. Observabilité (Prometheus + Grafana)

Consigne du formateur : Grafana réservé au monitoring technique ; la dataviz métier passe par un outil séparé (section suivante).

| Cible                             | Exportateur                                | Expose                                                                                                                                                                                            |
|-----------------------------------|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Conteneurs                        | docker-stats-exporter (custom, API Docker) | CPU/mémoire — remplace cAdvisor (échoue sous Docker Desktop Windows/Mac : conteneurs dans une VM cachée, /var/lib/docker monté ne correspond pas au vrai backend) et node-exporter (un seul hôte) |
| PostgreSQL<br>api-producer, spark | postgres-exporter<br>prometheus_client     | Connexions, disponibilité, taille<br>Métriques métier du pipeline                                                                                                                                 |

**Indicateur Raw vs Clean** (obligatoire) : raw\_records\_total (volume déposé dans raw/api/) vs clean\_records\_total (COUNT(\*) sur diagnostic\_dpe\_flux), tous deux recalculés depuis le volume persisté (pas un compteur remis à zéro au redémarrage). raw ≥ clean en permanence.

Dashboard TP2 — Pipeline temps réel provisionné automatiquement : 8 panels (Raw vs Clean, taux de nettoyage, débit du producteur, PostgreSQL, CPU/mémoire).

## 6. Data visualization

Outil dédié, distinct de Grafana (consigne du formateur) : Metabase, connecté à PostgreSQL — répartition des étiquettes DPE, communes les plus chères, part F-G par tranche de prix. Metabase n'ayant pas de mécanisme de provisioning par fichiers comme Grafana, [monitoring/metabase/bootstrap.py](#) configure compte ad-

min, connexion PostgreSQL et dashboard par l'API REST de Metabase, exécuté automatiquement au démarrage (admin@tp2.local / MetabaseTp2!2026 sur :3001) : aucune étape manuelle, y compris pour cet outil.

## 7. Vérification

Stack testée à froid (docker compose down -v && docker compose up -d) sur trois machines et réseaux différents, y compris reprise après un docker kill de Spark en plein flux (checkpoint Kafka : ni perte ni doublon).

| Étape                            | Résultat                                                                   |
|----------------------------------|----------------------------------------------------------------------------|
| Kafka (dpe-flux-44)              | Backfill complet : 351 959 événements publiés                              |
| PostgreSQL (diagnostic_dpe_flux) | 351 959 lignes après backfill complet                                      |
| Raw vs Clean                     | Vérifié à la source (fichiers + base) : raw ≥ clean à tout instant         |
| Prometheus                       | 4/4 cibles UP                                                              |
| Grafana                          | Dashboard monitoring provisionné ; requêtes vérifiées avec données réelles |

Le classement des communes les plus chères, recalculé en continu, retombe sur les valeurs du TP1 (La Baule-Escoublac 6 536 €/m², Pornichet 5 636 €/m², Nantes 3 777 €/m²).

### Comment reproduire

docker compose up -d

Démarre l'ensemble (TP1 + TP2) sans étape manuelle ni accès réseau obligatoire (le backfill utilise les CSV déjà versionnés).

### Comment vérifier

| Composant  | Commande                                                                                                                  |
|------------|---------------------------------------------------------------------------------------------------------------------------|
| Kafka      | docker exec tp2_kafka<br>/opt/kafka/bin/kafka-get-offsets.sh<br>--bootstrap-server localhost:19092 --topic<br>dpe-flux-44 |
| Data lake  | docker exec tp2_spark sh -c "wc -l<br>/datalake/raw/api/dpe_raw.ndjson<br>/datalake/aggregated/dpe_enrichi.ndjson"        |
| PostgreSQL | docker exec tp_audit_44_pg psql -U postgres -d<br>audit_immo_energie_44 -c "SELECT COUNT(*) FROM<br>diagnostic_dpe_flux;" |
| Prometheus | http://localhost:9090/targets                                                                                             |
| Grafana    | http://localhost:3000 (admin/admin)                                                                                       |
| Metabase   | http://localhost:3001 (admin@tp2.local /<br>MetabaseTp2!2026)                                                             |

### Ports exposés

| Port               | Service                                                              |
|--------------------|----------------------------------------------------------------------|
| 5432               | PostgreSQL                                                           |
| 9092               | Kafka (accès externe)                                                |
| 8001 / 8003 / 8004 | Métriques Prometheus : producteur / Spark /<br>docker-stats-exporter |
| 9187               | postgres-exporter                                                    |
| 9090               | Prometheus                                                           |
| 3000               | Grafana                                                              |
| 3001               | Metabase                                                             |

### Limites connues

Choix assumés, hors périmètre pour un TP sur une seule machine : data lake non partitionné par date, ON CONFLICT DO NOTHING (pas de mise à jour sur republication ADEME), Kafka mono-broker (réplication 1), pas de limites CPU/mémoire par conteneur.